

Stage 1 — Building the Data Layer

Technical Report on C++ Architecture, Implementation & Benchmarking Analysis

Course:	Big Data — B.Sc. Data Science and Engineering (ULPGC)	Date:	October 2026
Team:	TheScratchers	Branch:	ft/c++-setup (C++17)
Author:	Pablo Martinez Suarez	Repository:	github.com/TheScratchers/stage_1

1. Executive Summary & Specification Compliance

This report presents the complete architecture, implementation details, and empirical benchmark results of the **Data Layer (Stage 1)** developed in C++17 for the Project Gutenberg search engine. The system fully complies with all requirements outlined in the course specification (*stage_1_building_data_layer.pdf*) and adheres to the cross-language benchmark contract (*shared/CONTRACT.md*).

Requirement (Spec)	C++ Component	Implementation & Coverage
Ingestion & Header/Body Split	ControlLayer & Datalake	Delimits Gutenberg markers; partitions into .header.txt and .body.txt.
3 Datalake Layouts	Datalake (Time, Book, Batch)	Time-based (YYYYMMDD/HH), Book-based (id/), and Batch-based (batch_N/).
Metadata Datamart	MetadataExtractor (SQLite)	Structured schema with primary key and secondary indexes on author and title.
Metadata Benchmark (Page 5)	BenchmarkRunner (runMetadata)	Transactional batch inserts (>560k rows/s); ID lookups; Dickens/Carroll queries.
3 Inverted Indexes (Pages 6-8)	JsonIndex, Hierarchical, Binary	Monolithic JSON, Hierarchical folders, and custom Binary Compact (BIDX) format.
Standard Query Workload	words.txt (10 terms)	Evaluates frequent (time), medium (death, monster), and rare (wonderland, darcy) tiers.
Control Layer & State Tracking	ControlLayer (control/*.txt)	Tracks state in downloaded_books.txt and indexed_books.txt without duplicates.
Modularity & Clean Code	< 150 LOC Architecture	All .cpp source files are strictly constrained to under 150 lines of code.

2. System Architecture & Design Principles

The architecture enforces strict separation of concerns between raw unstructured storage (**Datalake**), optimized query engines (**Datamarts**), and pipeline orchestration (**Control Layer**). The implementation applies modern **C++17** idioms and Clean Code practices: every component is encapsulated in dedicated headers/sources, maintaining high cohesion and remaining strictly **under 150 LOC per file**.

- **Datalake Layer:** Serves as the repository for ingested texts. Exposes 3 interchangeable partitioning layouts evaluated under automated benchmarks.
- **Metadata Datamart:** Extracts title, author, and language from book headers via regular expressions, storing records in an indexed SQLite database.
- **Inverted Index Datamart:** Tokenizes body texts into lowercased alphabetic tokens (matching regex [a-zA-Z]+) and maintains postings across 3 distinct physical storage representations.
- **Control Layer:** Directs pipeline workflow, coordinates Gutenberg downloads via *libcurl*, and tracks progress

atomically.

3. Modular Codebase Breakdown (< 150 LOC per file)

Datalake (Datalake.hpp / Datalake.cpp - 94 LOC)

Manages raw document storage and layout resolution. Computes paths for TimeBased (YYYYMMDD/HH/), BookBased (ID/), and BatchBased (batch_N/, 500 books per partition). Provides saveBook, locateBook, and recursive search fallback.

MetadataExtractor (MetadataExtractor.hpp / MetadataExtractor.cpp - 127 LOC)

Parses title, author, and language using regular expressions. Implements saveBatchToDatabase using atomic SQLite transactions (BEGIN TRANSACTION / COMMIT) with prepared statement reuse, achieving ingestion throughputs exceeding 560,000 rows/second. Defines B-Tree indexes on author and title.

JsonIndex (JsonIndex.hpp / JsonIndex.cpp - 59 LOC)

Monolithic inverted index serialized using nlohmann/json. Provides index loading, disk serialization, and incremental term posting updates. Illustrates human-readable simplicity versus full-file rewrite overhead.

HierarchicalIndex (HierarchicalIndex.hpp / HierarchicalIndex.cpp - 60 LOC)

Sharded filesystem inverted index structured into alphabetical folders (A/ to Z/) containing individual <term>.txt files. Allows granular postings updates without touching unrelated vocabulary.

BinaryIndex (BinaryIndex.hpp / BinaryIndex.cpp - 132 LOC)

Custom high-performance binary index format. Features a 17-byte header (magic 'BIDX', uint8 version, uint32 term count, uint64 dictionary offset). Postings lists are stored as contiguous 32-bit unsigned integers, with a term dictionary table at the tail enabling direct seek-based lookup.

QueryEngine (QueryEngine.hpp / QueryEngine.cpp - 42 LOC)

Unified search facade supporting single-term queries across any index structure. Implements multi-term Boolean **AND** (postings intersection via std::set_intersection) and Boolean **OR** (postings union).

ControlLayer (ControlLayer.hpp / ControlLayer.cpp - 145 LOC)

Pipeline orchestrator. Reads and updates control/downloaded_books.txt and indexed_books.txt. Schedules pending downloads for indexing across all datamarts, and fetches new books via libcurl.

Tokenizer (Tokenizer.hpp / Tokenizer.cpp - 29 LOC)

Enforces the cross-language benchmark contract: lowercases input text and extracts maximal runs of alphabetic ASCII characters [a-zA-Z], dropping digits and punctuation without stemming or stopwords.

BenchmarkRunner (BenchmarkRunner.hpp / BenchmarkRunner.cpp - 130 LOC)

Automated benchmarking harness. Evaluates Datalake layouts (throughput, lookup latency, directory count), Metadata storage (batch transactions, ID lookups, author queries), and Inverted Indexes (build time, 10-word workload, update latency, disk footprint).

Unified CLI (main.cpp - 134 LOC)

Command-line entrypoint exposing: step, query, query-and, query-or, metadata, bench-datalake, bench-metadata, bench-index, bench-all, and test-sample.

4. Experimental Results & Benchmark Analysis

Benchmarks were executed on macOS (Apple Silicon M-series) using the **20 genuine Project Gutenberg books** mandated by the shared contract (including *Moby-Dick*, *Dracula*, *Frankenstein*, *Jane Eyre*, etc., comprising over 12 million characters of authentic literary prose).

4.1 Inverted Index Benchmark: Scalability & Trade-offs

Each index architecture was evaluated across synthetic corpus scales (100, 1,000, and 10,000 books). Query latency was measured using the 10 fixed contract search terms from *shared/words.txt* across all frequency tiers.

Structure	Scale (Books)	Build Time (s)	Lookup avg (ms)	Update Cost (ms)	Disk Footprint
json_monolithic	100	0.047 s	0.0018 ms	137.09 ms	8,867 KB (~8.8 MB)
hierarchical	100	3.524 s	0.1171 ms	0.45 ms	4,781 KB (~4.7 MB)
binary_compact	100	0.139 s	1.2711 ms	215.46 ms	3,472 KB (~3.4 MB)
json_monolithic	1,000	0.422 s	0.0014 ms	1,004.00 ms (1.0 s)	82,631 KB (~82 MB)
hierarchical	1,000	3.860 s	0.1635 ms	0.76 ms	47,809 KB (~47 MB)
binary_compact	1,000	0.198 s	1.0391 ms	315.67 ms	28,060 KB (~28 MB)
json_monolithic	10,000	4.736 s	0.0015 ms	11,187.30 ms (11.2 s)	820,269 KB (~820 MB)
hierarchical	10,000	7.722 s	0.9682 ms	3.92 ms	478,099 KB (~478 MB)
binary_compact	10,000	1.215 s	1.3350 ms	1,873.85 ms (1.8 s)	273,939 KB (~274 MB)

In-Depth Analysis & Trade-offs:

- **Monolithic JSON Write Bottleneck:** At 10,000 books, the JSON file reaches **820 MB**. Adding a single book requires rewriting the entire 820 MB file to disk, causing update latency to skyrocket to **11.18 seconds**. This validates the spec's warning: monolithic files are fundamentally unsuitable for continuous high-throughput ingestion.
- **Hierarchical Incremental Dominance:** The hierarchical structure updates in only **3.92 milliseconds** at 10,000 books—nearly **3,000 times faster** than JSON—because it only touches the files corresponding to words present in the added book.
- **Binary Compact Space Efficiency:** The binary index occupies **274 MB** compared to JSON's 820 MB (a **66% storage reduction**). It also achieves the fastest build time at scale (**1.21 s** vs 7.72 s for hierarchical) by eliminating filesystem inode overhead.

4.2 Metadata Storage Benchmark (SQLite Stress Test - Section 4.1)

Measures batch insertion speed and response times for `find_by_id` (primary key B-tree) and `find_by_author` comparing a repeated author (*Charles Dickens*, 30% of records) against a unique author (*Lewis Carroll*, 10% of records).

Scale (Books)	Insert Time (s)	Throughput (Rows/s)	ID Lookup (ms)	Dickens Lookup (ms)	Carroll Lookup (ms)	DB Size
100	0.0057 s	17,619 rows/s	0.165 ms	0.141 ms	0.119 ms	28 KB
1,000	0.0091 s	110,444 rows/s	0.127 ms	0.270 ms	0.194 ms	168 KB
10,000	0.0301 s	332,758 rows/s	0.083 ms	1.332 ms	0.911 ms	1.5 MB
100,000	0.1772 s	564,194 rows/s	0.070 ms	12.756 ms	8.766 ms	15.4 MB

Metadata Analysis:

- **Transaction Batching in Big Data:** By encapsulating inserts into an atomic transaction (`BEGIN ... COMMIT`), SQLite achieves a throughput of **564,194 rows/second**, storing 100,000 book records in just 0.17 seconds. Without transactions, naive autocommit would trigger 100,000 disk `fsync()` operations, taking over 2 minutes.
- **O(log N) vs O(N) Query Behavior:** Primary key lookups remain virtually instantaneous (0.07 ms) via B-Tree index

traversal. In contrast, author searches scale linearly with record volume (0.27 ms at 1k rows vs 12.75 ms at 100k rows), with Dickens queries taking longer due to the higher volume of matching records returned.

4.3 Datalake Layout Benchmark

Layout	Write Time (s)	Throughput (Books/s)	Lookup Latency (ms)	Directories Created
time_based (YYYYMMDD/HH)	0.1209 s	827.0 books/s	0.1690 ms	2 directories
book_based (ID/)	0.1325 s	754.4 books/s	0.0126 ms	100 directories
batch_based (batch_N/)	0.1323 s	755.8 books/s	0.0111 ms	1 directory

Datalake Conclusion: The *time_based* scheme minimizes directory inode contention during mass ingestion, while *book_based* and *batch_based* provide 10x faster direct path lookups without recursive filesystem scans.

5. Execution & Verification Guide

The C++ implementation provides a reproducible CMake build and a versatile CLI binary (search_engine):

```
# 1. Build the C++ executable
cd c++
cmake -B build -S . && cmake --build build

# 2. Offline sample dataset test
./build/search_engine test-sample

# 3. Inverted index queries (Single term / Boolean AND / Boolean OR)
./build/search_engine query truth all
./build/search_engine query-and truth fortune
./build/search_engine query-or wonderland darcy

# 4. Metadata queries (SQLite)
./build/search_engine metadata --id 1342
./build/search_engine metadata --author "Austen"

# 5. Automated Benchmarks
./build/search_engine bench-datalake 1000
./build/search_engine bench-metadata 10000
./build/search_engine bench-index 1000
./build/search_engine bench-all 1000
```